Sesion 5: DevOps + IA + Ejecucion en la nube

Sauce Labs · Azure Pipelines — las mismas pruebas, ahora en navegadores reales

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

1. Pre-requisitos
2. Puente Dia 4
3. Por que la nube
4. Sauce Labs
5. Portar a la nube
6. Pipeline de Azure
7. Multi-navegador y paralelismo
8. Reportes y evidencias
10. Troubleshooting
Ejercicios (Parte A)
9. IA en el CI/CD
Ejercicios IA (Parte B)
11. Riesgos auto-reparar
12. Healenium
Cierre

Guia rapida: que hacer para que TODO funcione

Resumen para dummies — hoy sumamos Sauce Labs y el ciclo romper/reparar

> ⚠️ Hoy partimos de lo del **Dia 4** (ya en verde) y agregamos: ejecucion en **Sauce Labs** (navegadores reales) y el ciclo **romper → detectar → reparar con IA**. Todo sigue corriendo en **Azure**; tu solo agregas **2 credenciales** de Sauce.

Pre-requisito: que lo del Dia 4 ya funcione

Antes de empezar, el pipeline del Dia 4 debe estar en verde: los 3 datos ajustados (tags `@TC` , `TEST_PLAN_ID` / `TEST_SUITE_ID`) y el PAT `AZURE_DEVOPS_PAT` como secreto. Hoy el proyecto `tienda-nube` ya trae eso mismo configurado.

Lo UNICO nuevo que agregas hoy: credenciales de Sauce

1. Crea cuenta en **saucelabs.com** (el trial sirve).
2. En Sauce: **Account** → **User Settings** → copia tu `Username` y tu `Access Key` .
3. En el pipeline: **Edit** → **Variables** → crea `SAUCE_USERNAME` y `SAUCE_ACCESS_KEY` como **secretas**.

Como la app llega a Sauce sin estar desplegada: el TUNEL

La app corre en el agente (`localhost:3000`) y Sauce vive en la nube. El pipeline abre un **tunel Sauce Connect** para que el navegador real de Sauce alcance esa app. **No despliegas nada**: si no defines las credenciales, el stage de Sauce se salta sin romper el pipeline.

🖍️ **EJERCICIO PRACTICO**

Ejercicio

El dia gira en torno a un ejercicio de regresion: **cambiamos la APLICACION** (como haria un desarrollador), vemos que la automatizacion lo detecta y lo reporta, y lo reparamos con ayuda de la IA. Se hace en dos pasadas: primero en local, luego en la nube (Sauce).

Que vamos a cambiar (la APP, no la prueba)

Importante: el cambio que rompe lo hacemos en la **aplicacion**, NO en la automatizacion. Es el caso real: un dev toca el front y, sin querer, rompe una prueba. La automatizacion queda igual; es su trabajo *detectar* ese cambio.

El cambio exacto: en la app, archivo `app-tienda/public/index.html` , el contenedor del catalogo tiene un identificador `data-testid="grid-productos"` . Lo cambiamos a `data-testid="grid-de-productos"` .

app-tienda/public/index.html — el identificador que cambiamos

```
<!-- ANTES (lo que la prueba espera) -->
<div id="grid" class="grid" data-testid="grid-productos"></div>

<!-- DESPUES (el cambio del dev que rompe la prueba) -->
<div id="grid" class="grid" data-testid="grid-de-productos"></div>
```

Al cambiar ese `data-testid`, la prueba de catalogo (`@TC20`) sigue buscando `grid-productos` y ya no lo encuentra → **falla con timeout**. Eso es lo que queremos: ver como el sistema lo detecta y reporta.

A **Local + Test Plan:** cambias el `data-testid` en la app → push → el test de catalogo falla → el **Test Plan** marca el caso 20 en **Failed** solo → diagnosticas con **Claude (MCP)** → reparas la app → verde.

B **Sauce + evidencia:** repites el cambio → el fallo aparece en **Sauce con video** → copias esa evidencia y se la pasas a **Claude** → reparas → verde en local y en Sauce.

💡 Donde esta el identificador que cambiamos: es un atributo `data-testid` en el HTML de la app (`app-tienda/public/index.html`). NO tocamos los `.feature` ni los `tests/`: la gracia es que la automatizacion detecte el cambio de la app.

📘 DOCUMENTACION DE REFERENCIA

Errores tipicos de Sauce (y su causa)

Sintoma	Causa	Arreglo
El stage de Sauce no corre	falta `SAUCE_USERNAME`	Define las 2 variables secretas
Timeout contra la app en Sauce	el tunel no quedo arriba o el nombre no coincide	Revisa `SAUCE_TUNNEL` (igual en el paso y en el conftest)
`unauthorized`	credenciales o region mal	Verifica Username/Access Key y la region

💡 En una frase: lo del Dia 4 debe estar verde; hoy agregas **2 credenciales de Sauce**, el pipeline abre el **tunel** solo, y practicas el ciclo **romper → detectar → reparar con IA**.

¿Que hace Claude y donde encaja?

La duda mas comun: que es Claude, que hace por mi y en que momento lo uso

⚠️ **Idea en una frase:** Claude vive en **TU maquina** y es tu **asistente para escribir y reparar pruebas**. **NO corre en el pipeline.** Azure ejecuta las pruebas; Claude te ayuda a crearlas y a entender por que fallan.

Que es Claude aqui (en cristiano)

Claude es un asistente de IA (como ChatGPT) que corre en una app de escritorio: **Claude Desktop**. Por si solo, Claude solo conversa. Lo potente es que le conectamos unas **"manos"**: el **servidor MCP de Playwright**, que le permite **abrir un navegador de verdad**, entrar a tu app, ver el contenido de la pagina (el DOM) y hacer clics. Asi Claude deja de solo "hablar" y puede **inspeccionar tu aplicacion** y ayudarte con datos reales, no suposiciones.



razona, propone, explica	abre el navegador, lee el DOM, hace clic	Claude la inspecciona de verdad

Que hace Claude POR TI (casos concretos del dia)

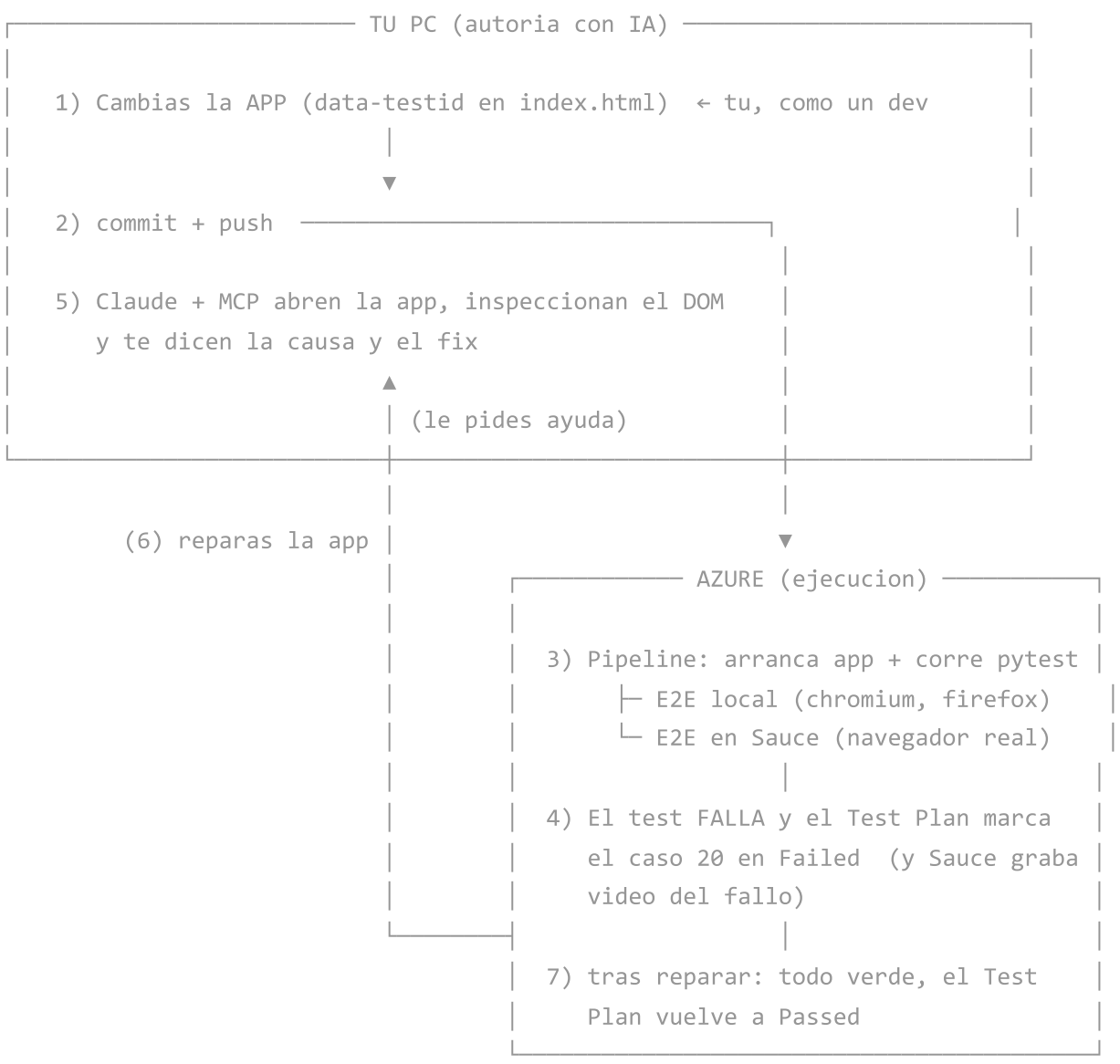
- **Inspeccionar:** "Abre la app y dime que `data-testid` tiene el contenedor del catalogo." → Claude entra, lo lee y te lo dice.
- **Diagnosticar un fallo:** "La prueba de catalogo falla con timeout buscando `grid-productos`. ¿Que paso?" → Claude compara lo que la prueba espera con lo que la app realmente tiene, y encuentra el desajuste.
- **Proponer el fix:** "¿Como lo arreglo?" → Claude te dice que el `data-testid` cambio y que hay que devolverlo (o actualizar el selector).
- **Crear pruebas nuevas:** "Genera un escenario Gherkin para buscar un producto por nombre." → Claude te da el `.feature` listo para revisar.
- **Analizar evidencia de la nube:** le pegas el log o el video de un fallo en Sauce y Claude te dice la causa probable.

Clave: Claude *propone*; **tu decides**, revisas y guardas el cambio. Claude acelera; no reemplaza tu criterio ni el framework.

MAPA: como se relaciona TODO el ejercicio

Este es el circuito completo del dia. Fijate que hay **dos zonas**: lo que pasa en TU PC (con Claude) y lo que pasa en AZURE (el pipeline). El ejercicio es un **ciclo** que las recorre.

El circuito completo (romper → detectar → reparar)



En palabras: tu cambias la app (1) y haces push (2). Azure corre las pruebas (3) y, como la app cambio, un test falla y el Test Plan lo reporta en rojo (4). Le pides a Claude que investigue (5), te dice la causa, reparas la app (6) y al volver a hacer push todo queda verde (7).

Que debes hacer con Claude (pasos reales)

1. Tener **Claude Desktop** instalado y logueado (cuenta Free), con el servidor MCP de Playwright registrado (lo hiciste en el **Dia 4**).
2. Tener la **app accesible** (levantada en tu PC con `npm start`) para que Claude la pueda "ver".
3. **Conversar** con Claude: pedirle que inspeccione, diagnostique un fallo o proponga un fix/escenario.
4. **Revisar** su propuesta, aplicar el cambio y hacer **push**. El pipeline de Azure hace el resto.

💡 Regla de oro: **Claude = autoria y diagnostico (en tu PC)**; **Azure = ejecucion y evidencia (en el pipeline)**. El registro paso a paso del MCP en Claude Desktop esta en el Dia 4; hoy solo lo usamos.

PARTE A

Sauce Labs: ejecutar las pruebas en la nube

La primera parte del dia es de **ejecucion**, SIN IA todavia. Tomamos el framework del Dia 4 y lo hacemos correr en **navegadores reales en la nube (Sauce Labs)**, integrado al pipeline de Azure, con evidencias (video, Allure, trazas). Objetivo: la MISMA suite, ahora tambien en la nube.

Cubre las secciones 1 a 8 · Ejercicios 5-8 (practica manual) · Aqui NO entra Claude todavia

1. Pre-requisitos de la sesion

Lo que necesitas listo antes de empezar

Herramienta	Version / plan	Para que	Como verificar
El framework del Dia 4	tienda-e2e funcionando	es la base: hoy lo llevamos a la nube	`pytest` en verde (local)
Node.js	18+ (20 recomendado)	correr la app Tienda	`node -v`
Python	3.10+ (probado 3.11)	correr las pruebas	`python --version`
Cuenta Sauce Labs	trial o pagada	ejecutar en navegadores reales (demo guiada)	login en app.saucelabs.com
Cuenta Azure DevOps	la del Dia 3	correr el pipeline con el stage de nube	dev.azure.com

El proyecto de hoy

Usamos `dia5/proyecto/tienda-nube` : es el mismo framework del Dia 4 (incluye su propia copia de `app-tienda/` + Screenplay + BDD, igual que el Dia 4) con dos piezas nuevas: un `conftest.py` que puede usar el navegador local **o** uno real en Sauce, y un `azure-pipelines.yml` que corre las pruebas en el pipeline. Es la **misma app Tienda** de siempre (mismos endpoints y puerto 3000); cada dia trae su copia para ser autonomo. No reescribimos pruebas: las movemos de entorno.

⚠️ **Honestidad real vs demo:** todo lo local corre de verdad, sin cuenta. Sauce Labs requiere cuenta (trial), por eso la ejecucion en la nube es **demo guiada** con la configuracion ya lista. El codigo esta preparado para conectarse.

2. Puente desde el Dia 4

De donde venimos y que agregamos hoy

Lo que ya teniamos

En el Dia 4 construimos un framework E2E con **Playwright + patron Screenplay + BDD (pytest-bdd)** en Python, automatizamos los 3 casos del Dia 3 y le sumamos IA con MCP + Claude. Las pruebas ya eran **portables por `BASE_URL`**: apuntaban a local o a la app desplegada cambiando una variable.

Lo que agregamos hoy

- **Portabilidad del navegador:** las mismas pruebas corren en tu Chromium local o en **Chrome/Edge reales en Sauce Labs**, cambiando solo variables de entorno.
- **Integracion CI/CD:** metemos las pruebas al **pipeline de Azure**, con un stage local y uno en la nube.
- **Evidencias:** Sauce graba video, logs y capturas de cada corrida, utiles para auditar y depurar.

Aspecto	Dia 4	Dia 5
Navegador	Chromium local	Local **o** Chrome/Edge reales en Sauce

Aspecto	Dia 4	Dia 5
Donde corre	Tu maquina	Tu maquina **y** el pipeline de Azure
Seleccion de entorno	—	Variables de entorno (`SAUCE=1`), sin tocar el codigo
Evidencia	Salida de pytest	Video + logs + capturas en el dashboard de Sauce

3. Por que ejecutar en la nube

Lo que tu Chromium local no puede darte

Compatibilidad real

Safari, Edge, Chrome en Windows/macOS y navegadores moviles reales. Tu maquina solo tiene uno o dos.

Paralelismo a escala

Decenas de pruebas en paralelo en VMs de la nube. Lo que local tarda una hora, alla son minutos.

Evidencia automatica

Video, logs de red y capturas por cada corrida, sin configurar nada. Clave para auditar fallos.

Ambiente limpio

Cada corrida usa una VM nueva. Adios "en mi maquina si funciona" por estado residual.

El principio: una suite, muchos entornos

No se trata de reescribir pruebas para la nube. Se trata de que **la misma suite** corra donde la necesites: en tu maquina mientras desarrollas, y en navegadores reales cuando validas un release. El framework ya esta disenado para eso (localizadores `data-testid`, portabilidad por `BASE_URL`).

4. Sauce Labs: la nube de navegadores

Que es, como se crea la cuenta y donde estan las credenciales

Que es Sauce Labs

Una plataforma que te presta **navegadores y dispositivos reales en la nube** para correr tus pruebas automatizadas y manuales. En vez de mantener un laboratorio de equipos, pides "Chrome en Windows 11" o "Safari en iPhone" y Sauce te lo da por unos minutos, con video y logs de todo lo que paso.

✏️ EJERCICIO PRACTICO

Ejercicio 1: crear la cuenta y obtener las credenciales

1. Entra a `saucelabs.com` → **Sign up** (hay prueba gratuita). Confirma tu correo e inicia sesion en `app.saucelabs.com`.

2. Ve a **Account** → **User Settings**. Ahi veras tu **Username** y tu **Access Key** (el access key se copia con un boton; tratalo como una contrasena).

3 Anota tambien tu **Data Center / region** (ej. `us-west-1` o `eu-central-1`). Lo necesitas para apuntar al endpoint correcto.

⚠️ **Seguridad:** el SAUCE_ACCESS_KEY es como una contrasena. NUNCA lo subas al repositorio ni lo escribas en el YAML. Usa variables de entorno en local y **variables secretas** en el pipeline.

📘 DOCUMENTACION DE REFERENCIA

Dos formas de ejecutar Playwright en Sauce

- **saucectl (CLI de Sauce):** empaqueta tu proyecto Playwright y lo corre en la nube segun un `.sauce/config.yml`. Es la via nativa para Playwright **JS/TS**.
- **Conexion remota (la que usamos):** como nuestro framework es Playwright for **Python**, conectamos Playwright al navegador remoto de Sauce desde `conftest.py`. Ambas corren en la nube de Sauce; cambia como se lanzan.

Fuentes oficiales (Sauce Labs): Playwright en Sauce · Config de saucectl · Conexion remota.
Contenido reformulado para cumplir con las restricciones de licencia.

5. Portar las pruebas a la nube (sin reescribirlas)

El conftest decide: navegador local o navegador real en Sauce

La idea clave

El actor del patron Screenplay recibe una `page` de Playwright. Da igual si esa page es un Chromium local o un Chrome real en Sauce: las tasks, questions y features son **identicas**. Lo unico que cambia es de donde sale la page, y eso lo decide el `conftest.py` segun variables de entorno.

conftest.py — el actor elige el navegador por entorno

```
def _usar_sauce() -> bool:
    return os.environ.get("SAUCE") == "1" and bool(
        os.environ.get("SAUCE_USERNAME") and os.environ.get("SAUCE_ACCESS_KEY")
    )

@pytest.fixture
def actor(request, base_url):
    if _usar_sauce():
        pagina = request.getfixturevalue("page_sauce")   # Chrome real en Sauce
    else:
        pagina = request.getfixturevalue("page")         # Chromium local
    return Actor.llamado("Ana").que_puede(
        NavegarConElNavegador.usando(pagina, base_url))
```

🏏 EJERCICIO PRACTICO

Ejercicio 2: correr la MISMA suite en local y en Sauce

1 **Local** (como el Dia 4): con la app en `localhost:3000`, ejecuta `pytest`. Usa tu Chromium.

2 **En Sauce:** define las credenciales y apunta `BASE_URL` a la app PUBLICA (Sauce no ve tu `localhost`). Luego `pytest`.

3 Abre el dashboard de Sauce: veras la corrida con video y logs. La misma prueba, otro navegador, otra nube.

PowerShell — ejecutar en Sauce Labs

```
$env:SAUCE="1"
$env:SAUCE_USERNAME="<tu-usuario>"
$env:SAUCE_ACCESS_KEY="<tu-access-key>"
$env:SAUCE_REGION="us-west-1"          # o eu-central-1
$env:SAUCE_BROWSER="chrome"            # chrome | MicrosoftEdge
$env:SAUCE_PLATFORM="Windows 11"
$env:BASE_URL="https://<tu-app-publica>"
pytest

# Al terminar, limpia las variables:
Remove-Item Env:\SAUCE
```

⚠️ Sauce corre en la nube: necesita una **URL publica** de la app (App Service / App Runner), no `localhost`. Si la app es interna, se usa un tunel con **Sauce Connect**.

6. Integrar al pipeline de Azure

Las pruebas corren solas en cada push: primero local, luego en la nube



Como esta armado el azure-pipelines.yml (3 stages)

- **Build:** instala la app (Node + Faker) y las dependencias Python + el navegador de Playwright.
- **E2E_Local:** corre las pruebas en una **maquina virtual Ubuntu que Azure crea solo para este run** (el "agente hosted", `ubuntu-latest`). Esa VM arranca la app en SU `localhost:3000`, corre `pytest` con el navegador **instalado en esa misma VM** (Chromium/Firefox) y publica JUnit. Es rapido y gratis, pero el navegador NO es "real": es el motor de Playwright dentro de Linux. Al terminar el run, Azure **destruye la VM**.
- **E2E_Sauce:** corre las MISMAS pruebas, pero el **navegador es real y vive en la nube de Sauce Labs** (ej. Chrome en Windows 11), no en la VM de Azure. La VM de Azure solo sostiene la app y el tunel; el navegador que "pulsa botones" esta en Sauce, y queda grabado en **video**. Solo se activa si existen `SAUCE_USERNAME` y `SAUCE_ACCESS_KEY` (si no, el pipeline salta este stage sin fallar).

La diferencia en una tabla

	E2E_Local	E2E_Sauce
Donde corre el navegador	en la VM Ubuntu de Azure	en la nube de Sauce (VM real)
Que navegador	Chromium/Firefox (motor Playwright en Linux)	Chrome/Edge/Safari REALES, sobre Windows/Mac
Donde vive la app	localhost de la VM de Azure	localhost de la VM de Azure (Sauce lo ve por el tunel)
Evidencia extra	JUnit + Allure + trace	+ video de la sesion
Velocidad / costo	rapido, gratis	mas lento, consume minutos de Sauce

La app NO es publica: entra Sauce Connect (el tunel)

Sauce Labs corre los navegadores en la nube y **no puede ver tu** `localhost` . Como nuestra app no esta desplegada con una URL publica, usamos **Sauce Connect**: un tunel que expone el `localhost:3000` del agente a los navegadores reales de Sauce. Asi la MISMA URL funciona en local y en la nube, sin desplegar nada.



azure-pipelines.yml (stage de Sauce con tunel)

```
- stage: E2E_Sauce
  dependsOn: E2E_Local
  condition: and(succeeded(), ne(variables['SAUCE_USERNAME'], ''))
  jobs:
    - job: e2e_sauce
      steps:
        - script: node servidor/server.js &          # 1) app en el agente
          workingDirectory: app-tienda
        - script: |                                   # 2) abrir el tunel
            ./sauce-connect run --username "$SAUCE_USERNAME" \
              --access-key "$SAUCE_ACCESS_KEY" --region us-west-1 \
              --tunnel-name "$SAUCE_TUNNEL" &
          env: { SAUCE_USERNAME: $(SAUCE_USERNAME), SAUCE_ACCESS_KEY: $(SAUCE_ACCESS_KEY), SAUCE_TUNNEL: 'tienda-
tunnel-$(Build.BuildId)' }
        - script: pytest --junitxml=resultados/junit-sauce.xml   # 3) navegador real
          env:
            SAUCE: '1'
            SAUCE_USERNAME: $(SAUCE_USERNAME)      # variable SECRETA
            SAUCE_ACCESS_KEY: $(SAUCE_ACCESS_KEY)  # variable SECRETA
            SAUCE_TUNNEL: 'tienda-tunnel-$(Build.BuildId)'
            BASE_URL: 'http://localhost:3000'      # Sauce la ve por el tunel
```

✏️ **EJERCICIO PRACTICO**

Ejercicio 3: configurar las credenciales como secretos

1. En Azure DevOps: **Pipelines → (tu pipeline) → Edit → Variables**.

2. Crea `SAUCE_USERNAME` y `SAUCE_ACCESS_KEY` marcando **Keep this value secret** (candado). NO necesitas `BASE_URL` publica: la app se alcanza por el tunel en `localhost:3000` .

3. Guarda y haz un **Run**. El stage E2E_Sauce abrira el tunel, correra en un navegador real y veras la corrida (con video) en el dashboard de Sauce.

✏️ **EJERCICIO PRACTICO**

Ejercicio 3b: crear el tunel de Sauce Connect (paso a paso)

El tunel es lo que permite que un navegador en la nube de Sauce vea tu app en `localhost`. En el pipeline se abre solo (ya esta en el YAML); aqui ves los pasos para entenderlo y, si quieres, abrirlo en tu PC.

Que es (en una frase)

Sauce Connect es un programa que crea un "cable" seguro entre tu maquina (donde esta la app en `localhost:3000`) y la nube de Sauce. Mientras ese cable esta abierto, el navegador real de Sauce puede abrir `http://localhost:3000` como si estuviera en tu red.

1. **Descarga Sauce Connect** desde el sitio de Sauce (seccion Tunnels / Downloads). Es un ejecutable; no requiere instalacion.

2 **Ten la app corriendo** en `localhost:3000` (en otra terminal: `npm start` desde `app-tienda`).

3 **Abre el tunel** con tu usuario, access key, region y un nombre de tunel (el nombre lo eliges tu; debe coincidir con el que usa la prueba).

4 **Espera a que diga "ready"/"up".** El tunel queda abierto en esa terminal; dejala corriendo mientras pruebas.

5 **Corre las pruebas** con `SAUCE=1` , `SAUCE_TUNNEL=<mismo-nombre>` y `BASE_URL=http://localhost:3000` . El navegador de Sauce alcanzara tu app por el tunel.

6 **Al terminar, cierra el tunel** (Ctrl+C en su terminal). En el pipeline, el agente efimero se destruye y el tunel se cierra solo.

PowerShell — abrir el tunel en tu PC (opcional, para probar en local)

```
# 1) En una terminal: la app corriendo
#    (desde app-tienda)  npm start      # http://localhost:3000

# 2) En otra terminal: abrir el tunel (ajusta la ruta al ejecutable descargado)
.\sauce-connect.exe run `
  --username "<tu-usuario>" `
  --access-key "<tu-access-key>" `
  --region us-west-1 `
  --tunnel-name "tienda-tunnel"
#   Espera a ver el mensaje de que el tunel esta listo (ready/up).

# 3) En una tercera terminal: correr las pruebas por el tunel
$env:SAUCE="1"
$env:SAUCE_USERNAME="<tu-usuario>"
$env:SAUCE_ACCESS_KEY="<tu-access-key>"
$env:SAUCE_TUNNEL="tienda-tunnel"       # MISMO nombre que arriba
$env:BASE_URL="http://localhost:3000"   # Sauce la ve por el tunel
pytest
Remove-Item Env:\SAUCE
```

En el **pipeline** todo esto ya esta automatizado: un paso descarga Sauce Connect, otro abre el tunel con `--tunnel-name tienda-tunnel-$(Build.BuildId)`, y el `conftest.py` pasa ese mismo nombre como `tunnelName`. No tienes que hacerlo a mano salvo que quieras probar en tu PC.

💡 El nombre del tunel (`SAUCE_TUNNEL`) debe coincidir en el paso que lo abre y en las capabilities de la prueba (lo pasa el `conftest.py` como `tunnelName`). Usamos el BuildId para que cada run tenga su propio tunel.

💡 El stage de Sauce tiene `condition: ne(variables['SAUCE_USERNAME'], '')`: si nadie definio el usuario, el pipeline salta ese stage sin fallar. Asi la clase que no tiene cuenta igual ve el pipeline en verde con el E2E local.

📘 DOCUMENTACION DE REFERENCIA

Publicar resultados y proteger secretos

- **PublishTestResults@2:** sube los resultados (formato JUnit) para que Azure los muestre junto a los Test Cases, como en el Dia 4.
- **Variables secretas:** Azure las cifra y no las imprime en los logs. Es la forma correcta de pasar el access key de Sauce al pipeline.
- **Condiciones de stage:** permiten activar el stage de nube solo cuando hay credenciales, manteniendo el pipeline util para todos.

Fuentes oficiales (Microsoft Learn): Variables y secretos · Condiciones de stage · PublishTestResults@2.
Contenido reformulado para cumplir con las restricciones de licencia.

7. Matriz multi-navegador y paralelismo

Probar en varios navegadores y correr muchas pruebas a la vez

Cross-browser: el mismo caso en varios motores

Playwright trae tres motores de navegador: **Chromium** (Chrome/Edge), **Firefox** y **WebKit** (el motor de Safari). Un bug puede aparecer solo en uno. Correr la suite en los tres es la **matriz de compatibilidad**, y lo mejor: no reescribes nada, solo cambias el navegador. Esto corre **en local, sin cuenta de nube**.

PowerShell — instalar y correr en los 3 motores

```
python -m playwright install firefox webkit   # chromium ya estaba
pytest --browser chromium
pytest --browser firefox
pytest --browser webkit                        # motor de Safari
```

⚠️ Con pytest-bdd, `--browser chromium --browser firefox` en una sola corrida NO duplica los escenarios (es una limitacion conocida del plugin). Por eso corremos **un navegador por corrida**, o mejor aun, lo paralelizamos con una **matriz en el pipeline** (un job por navegador).

Paralelismo: mas pruebas en menos tiempo

Con **pytest-xdist** repartes las pruebas entre varios procesos (workers). En local aprovecha tus nucleos; en la nube, decenas de VMs. Lo que en serie tarda minutos, en paralelo son segundos.

PowerShell — ejecucion en paralelo

```
pytest -n auto      # usa todos los nucleos disponibles
pytest -n 4         # 4 workers en paralelo
```

azure-pipelines.yml — matriz de navegadores (extracto)

```
jobs:
  - job: e2e_local
    strategy:
      matrix:
        chromium: { BROWSER: 'chromium' }
        firefox:  { BROWSER: 'firefox' }
      maxParallel: 2
    steps:
      - script: pytest -n auto --browser $(BROWSER) --junitxml=resultados/junit-$(BROWSER).xml
```

💡 La matriz del pipeline corre un job completo por navegador, en paralelo. Es la version "en serio" de lo que haces a mano con `--browser`: cobertura cross-browser automatica en cada push.

📘 DOCUMENTACION DE REFERENCIA

Multi-navegador y paralelo — referencia

- **Motores de Playwright:** chromium, firefox y webkit se instalan con `playwright install` y se eligen con `--browser`.
- **pytest-xdist:** `-n auto` reparte las pruebas entre procesos; ideal para suites grandes.
- **strategy: matrix (Azure):** clona un job por cada combinacion (aqui, por navegador) y los corre en paralelo con `maxParallel`.

Fuentes: Playwright pytest (--browser) · pytest-xdist · Jobs y matrix (Azure).
Contenido reformulado para cumplir con las restricciones de licencia.

8. Reportes y evidencias

Que paso, por que fallo y como demostrarlo

Por que no basta "4 passed"

En un pipeline, la salida de texto de pytest es poca evidencia. Cuando algo falla a las 3am necesitas: que paso exactamente, en que paso, con que datos, y a ser posible un **video** o una **traza**. Para eso hay tres niveles de evidencia que usamos hoy.

Nivel	Herramienta	Que te da
JUnit XML	`--junitxml`	resultados que Azure publica junto a los Test Cases (como el Dia 4)
Reporte Allure	`allure-pytest`	reporte HTML con pasos, adjuntos, historico y graficos de tendencia
Trace de Playwright	`--tracing`	linea de tiempo con snapshots del DOM, red y consola; se abre en Trace Viewer
Video (en Sauce)	Sauce Labs	grabacion de la corrida en el navegador real

✏️ EJERCICIO PRACTICO

Ejercicio 5: generar un reporte Allure y una traza

1. Corre con Allure: `pytest --alluredir=resultados/allure` . Se crean los resultados en esa carpeta.
2. Captura una traza para depurar: `pytest --tracing on` . Playwright guarda un `trace.zip` por prueba en `test-results/` .
3. Abre la traza en el visor: `python -m playwright show-trace <ruta>\trace.zip` . Veras la linea de tiempo con cada accion.

PowerShell — reportes y traza

```
pytest --alluredir=resultados/allure         # resultados Allure
pytest --tracing on                          # genera trace.zip por prueba
python -m playwright show-trace test-results\...\trace.zip

# Reporte HTML de Allure (requiere el CLI de Allure, opcional):
# allure serve resultados/allure
```

💡 El **Trace Viewer** de Playwright es oro para depurar: ves el estado del DOM en cada paso, lo que la prueba "veia" cuando fallo. Mucho mas util que leer un stacktrace.

📘 DOCUMENTACION DE REFERENCIA

Reportes y trazas — referencia

- **Allure:** `allure-pytest` escribe resultados por prueba; el CLI de Allure los convierte en un reporte HTML navegable con historico.
- **Trace de Playwright:** `--tracing on` (o `retain-on-failure`) graba una traza; `show-trace` la abre con snapshots de DOM, red y consola.
- **En CI:** publica los resultados con `PublishTestResults@2` y adjunta Allure/trazas como artefactos del pipeline.

Fuentes: Trace Viewer · Allure + pytest · Artefactos de pipeline.
Contenido reformulado para cumplir con las restricciones de licencia.

10. Troubleshooting (lo que mas falla)

Errores tipicos y como resolverlos rapido

Sintoma	Causa probable	Solucion
Timeout / "no se pudo conectar" en Sauce	`BASE_URL` apunta a `localhost`	Sauce no ve tu maquina: usa una URL publica o Sauce Connect
"Unauthorized" en Sauce	Username/Access Key mal o region equivocada	Revisa Account → User Settings y `SAUCE_REGION`
El pipeline no corre el stage de Sauce	Falta la variable `SAUCE_USERNAME`	Es a proposito: el stage se salta sin credenciales. Definelas como secreto
`--browser firefox` falla al iniciar	Navegador no instalado	`python -m playwright install firefox webkit`
`-n auto` no paraleliza	Falta pytest-xdist	`pip install -r requirements.txt` (ya lo incluye)
El access key aparece en los logs	Variable normal en vez de secreta	Marca "Keep this value secret" en el pipeline; nunca en el YAML
Reporte Allure vacio	Olvidaste `--alluredir`	Corre `pytest --alluredir=resultados/allure`

💡 Regla de oro con la nube: si funciona en local pero no en Sauce, el 90% de las veces es la **URL** (localhost) o las **credenciales/region**. Empieza por ahi.

Ejercicios: practica guiada

Correr, comparar y evidenciar

⚠️

¿Donde se ejecuta cada ejercicio? (leer primero)

Hay dos tipos de ejercicio. Los de **practica manual** los corres TU en tu terminal (`pytest ...`) para APRENDER a usar la herramienta. Los de **ciclo completo** usan el **pipeline de Azure** (push) + Claude en tu PC. No se contradicen: primero practicas a mano para entender, luego el pipeline lo hace automatico.

Ejercicio	Donde se ejecuta	Para que
5 — Allure + trazas	TU PC (terminal, `pytest`)	ver que evidencia se genera
6 — provocar fallo + triaje IA	TU PC (terminal + Claude)	practicar el triaje con IA
7 — cross-browser	TU PC (terminal)	ver la suite en varios navegadores
8 — local vs nube	TU PC + Sauce (a mano)	comparar los dos entornos
9 — romper/reparar (local)	**Pipeline Azure** + Claude en tu PC	el ciclo real de regresion
10 — romper/reparar (Sauce)	**Pipeline Azure + Sauce** + Claude en tu PC	el ciclo con evidencia de nube

En una frase: ejercicios 5-8 = aprendes a mano en tu PC. Ejercicios 9-10 = lo pones a correr en el pipeline y usas Claude para diagnosticar. Lo que practicas a mano es lo mismo que el pipeline automatiza.

✏️ EJERCICIO PRACTICO

Ejercicio 7: cross-browser en local (en TU PC, a mano)

1. Instala los motores: `python -m playwright install firefox webkit` .
2. Corre la suite en los tres: `pytest --browser chromium` , luego `firefox` , luego `webkit` . Compara que todos pasen.
3. Mide el tiempo con y sin paralelismo: `pytest` vs `pytest -n auto` .

🏏 **EJERCICIO PRACTICO**

Ejercicio 8: comparar local vs nube con el mismo caso

1. Corre `pytest tests/test_catalogo.py` en local y observa el tiempo.
2. Si tienes cuenta Sauce, corre el mismo con `SAUCE=1` apuntando a un navegador distinto (`SAUCE_BROWSER="MicrosoftEdge"`). Compara: mismo resultado, otro navegador.
3. Abre el video en el dashboard de Sauce. Explica que agrega esa evidencia frente a solo la salida de pytest.

💡 Las mismas pruebas, sin un solo cambio de codigo, corrieron en dos navegadores y dos entornos. Esa portabilidad es el objetivo del dia. Con esto cierra la **Parte A** (ejercicios 5-8, practica manual). Lo que sigue es la **Parte B**: la IA.

PARTE B

IA: diagnosticar y reparar con Claude

Ya tenemos las pruebas corriendo en local, pipeline y Sauce (Parte A). Ahora entra la **IA**: cuando algo falla, usamos **Claude + MCP** para diagnosticar la causa en segundos. Es el ciclo **romper → detectar → diagnosticar con IA → reparar**. Importante: la IA **aconseja**; TU decides y aplicas el cambio.

Cubre: triaje de fallos (seccion 9) · Ejercicios 9-10 (ciclo romper/reparar) · Riesgos de la autorreparacion (seccion 11)

9. IA en el CI/CD: triaje de fallos

Que la IA haga el primer diagnostico cuando el pipeline se pone rojo

El problema que resuelve

Cuando el pipeline falla, alguien tiene que abrir los logs, entender que se rompio y decidir si es un bug, un selector fragil o un problema de entorno. Eso cuesta tiempo. La idea del dia: que el pipeline genere un **resumen estructurado del fallo** y un **prompt listo para la IA** (Claude + MCP del Dia 4), que da el primer diagnostico.

Como funciona nuestro triaje

El script `tools/triage_ia.py` lee el reporte JUnit, resume pasadas/fallidas y, por cada fallo, arma un prompt que le pide a Claude: (1) la causa raiz probable, (2) si es bug / selector / entorno, y (3) un arreglo concreto. Como Claude tiene el **MCP de Playwright**, puede incluso abrir la app y verificar el selector sospechoso.

PowerShell — triaje asistido por IA

```
pytest --junitxml=resultados/junit.xml     # 1) genera el reporte
python tools/triage_ia.py                  # 2) resume y arma el prompt
```

Salida (ejemplo con un fallo)

```
-- Fallos detectados ------------------------
  1. tests.test_catalogo::test_filtrar_productos_por_categoria_hogar
     [failure] TimeoutError: locator.click: Timeout 10000ms ...
-- PROMPT PARA CLAUDE (copia y pega) --------
Eres un ingeniero de QA. Estas pruebas E2E ... fallaron en el pipeline.
Para cada fallo: 1) causa raiz, 2) bug/selector/entorno, 3) arreglo concreto.
Usa el servidor MCP de Playwright para verificar el selector si hace falta.
...
```

✏️ **EJERCICIO PRACTICO**

Ejercicio 6: provocar un fallo y triarlo con IA

1. Rompe a proposito un selector en una Task (ej. cambia `agregar-1` por `agregar-99`) y corre `pytest --junitxml=resultados/junit.xml`. Veras el fallo.
2. Ejecuta `python tools/triage_ia.py`: genera el resumen y el prompt (tambien lo guarda en `resultados/triage-prompt.txt`).
3. Pega el prompt en Claude (con el MCP del Dia 4 activo). Pidele que verifique el selector contra la app y proponga el arreglo. Revisa su diagnostico.
4. Revierte el cambio y confirma que vuelve a verde. Reflexiona: la IA te dio el primer diagnostico; tu decides y versionas.

⚠️ En el pipeline, el paso de triaje corre con `condition: always()`: si hay fallos, deja el prompt como artefacto. La IA **acelera el diagnostico**, no decide por ti. El humano revisa y arregla.

Donde encaja en el pipeline

En el `azure-pipelines.yml`, despues de correr `pytest`, un paso ejecuta `tools/triage_ia.py` sobre el JUnit y publica el prompt como artefacto. Asi, quien revise el pipeline rojo encuentra ya el diagnostico listo para la IA, sin bucear en logs crudos.

Ejercicios de IA: el ciclo romper → reparar

El corazon de la Parte B: la IA diagnostica, tu reparas

⚠️ **El ejercicio estrella de hoy: romper → detectar → reparar con IA**

Hasta aqui todo pasa en verde. Ahora el ciclo real de QA: introducimos un **cambio que rompe** la app, dejamos que el pipeline lo **detecte** (y lo reporte en el Test Plan), usamos la **IA (MCP)** para diagnosticar y **reparar**, y confirmamos el verde. Primero en local, luego en Sauce con sus evidencias en video.

✏️ **EJERCICIO PRACTICO**

Ejercicio 9 — Romper, detectar en Test Plan y reparar con MCP (local)

El ciclo completo de regresion asistida por IA, sobre lo que ya construiste en el Dia 4.

1. **Rompe la app a proposito.** En `app-tienda/public/index.html`, cambia el `data-testid` del grid: de `grid-productos` a `grid-de-productos`. Es el tipo de cambio real que hace un dev sin avisar a QA.
2. **Commit + push.** El pipeline corre y el escenario de catalogo (`@TC20`) **FALLA**: el test espera `[data-testid="grid-productos"] .card` y da timeout.
3. **El Test Plan lo reporta solo.** En Azure Test Plans → Execute, el Test Case 20 pasa de Passed a **Failed** (lo publica `publicar_test_plan.py` con el `test_case_id` del tag).
4. **Diagnostica con Claude (EN TU PC).** Levanta la app (`npm start`) y en Claude: "Abre la home, el grid de productos no carga. Inspecciona el DOM y dime que `data-testid` tiene el contenedor y cual espera la prueba." → Claude **te DICE** la causa (no toca tu codigo).
5. **Reparas TU (no Claude).** Claude te dijo que el `data-testid` cambio. **Tu** abres el archivo `index.html`, devuelves `grid-productos` y guardas. La IA diagnostica; **el cambio al codigo lo haces tu**. Luego commit + push.
6. **Confirma el verde.** El pipeline vuelve a correr: 4/4 en verde y el Test Case 20 vuelve a **Passed**. Ciclo cerrado.

⚠️ **¿Donde pasa cada paso?** Pasos 1, 2, 5, 6 (editar la app, push) = en tu editor/repo. Pasos 2-3 (ejecutar y reportar) = en el **pipeline de Azure**, automatico. Paso 4 (diagnosticar) = **Claude en TU PC**. OJO: Claude NO repara; Claude DIAGNOSTICA. La reparacion (paso 5) la haces TU editando el archivo.

💡 Lo valioso: el pipeline detecto el fallo y el Test Plan lo reporto en rojo SOLOS. La IA ayudo a ubicar la causa en segundos. Pero el fix lo decides y lo aplicas TU: QA como red de seguridad, con el humano al mando.

🪛 **EJERCICIO PRACTICO**

Ejercicio 10 — Lo mismo en Sauce Labs: el MCP revisa la evidencia de la nube

Ahora el fallo ocurre en un navegador real de la nube, y usamos las evidencias de Sauce (video, screenshots, logs) como insumo para la IA.

1. **Activa el stage de Sauce.** Define los secretos `SAUCE_USERNAME` y `SAUCE_ACCESS_KEY` en el pipeline. El stage `E2E_Sauce` abre el **tunel Sauce Connect** y corre las pruebas en un navegador real contra `localhost:3000` (via tunel, porque la app no es publica).
2. **Rompe de nuevo.** Repite el cambio del grid (o rompe otro, ej. el `data-testid="agregar-1"` del boton). Push.
3. **El fallo aparece en Sauce.** Abre el dashboard de Sauce Labs → el build `tienda-e2e-<id>` → el test en rojo. Ahi tienes **video de la sesion**, screenshots paso a paso y los logs del navegador. Evidencia que pytest solo no da.
4. **Pasa la evidencia a Claude (EN TU PC).** Descarga el video o copia el log/error de Sauce y pegaselo a Claude: "Esta prueba fallo en Sauce. Este es el log y la captura del fallo. ¿Cual es la causa y el fix?" → Claude **analiza y te PROPONE** el arreglo.
5. **Reparas TU y confirmas.** Con lo que Claude te propuso, **tu** aplicas el fix en el archivo, guardas y haces push. Verifica el verde en el stage local y en el de Sauce. (Claude propuso; el cambio lo hiciste tu.)

⚠️ **¿Donde pasa cada paso?** Romper y reparar (1, 2, 5) = en tu editor/repo. Ejecutar y grabar video (2-3) = **pipeline de Azure + Sauce**, automatico. Diagnosticar (4) = **Claude en TU PC**, con la evidencia que TU le llevas de Sauce. Igual que antes: Claude diagnostica, TU reparas.

⚠️ **Como "ve" el MCP la evidencia de Sauce (honesto):** el servidor MCP de Playwright maneja un navegador, NO la API de Sauce. Entonces el flujo real es: TU traes la evidencia de Sauce (video/log/screenshot) y se la das a Claude en el chat; Claude la analiza y propone el fix. No se conecta solo a Sauce. Es igual de util y es lo que de verdad funciona hoy.

El ciclo que acabas de dominar

Cambio rompe → pipeline detecta → Test Plan/Sauce reportan → IA diagnostica con la evidencia → reparas → verde. Local para rapidez, Sauce para fidelidad (navegadores reales + video). La IA acelera el diagnostico en ambos.

11. Riesgos de dejar que la IA repare SOLA

Por que en este curso la IA diagnostica pero el humano decide

De que hablamos

"Autorreparacion automatica" (self-healing total) seria que, al fallar una prueba, un agente de IA **edite el codigo, haga commit y lo suba SIN que un humano revise**. Suena ideal, pero en QA tiene riesgos serios. Por eso en nuestro laboratorio la IA **propone** y el humano **aprueba**. Aqui vemos por que esa frontera importa.

Riesgo	Que puede pasar	Como lo mitigamos
Enmascarar un bug real	La IA "arregla" el selector para que la prueba pase... pero el boton de verdad se rompio. La prueba queda verde y el bug llega a produccion.	El humano revisa SI el cambio es un fix legitimo o esta tapando un defecto.
Falsos verdes	La prueba se auto-ajusta tanto que ya no valida lo que debia. Pierde su valor como red de seguridad.	Revisar que el escenario siga probando la intencion de negocio.
Cambios sin trazabilidad	Un commit automatico de la IA sin contexto: nadie sabe por que cambio el selector ni si fue correcto.	El fix pasa por PR / revision humana con explicacion.
Deriva silenciosa	Muchas auto-reparaciones acumuladas alejan las pruebas de la realidad del producto, sin que nadie lo note.	Auditar periodicamente que "sano" y por que.
Seguridad / permisos	Un agente con permiso de escribir en el repo y hacer push es una superficie de ataque y de error.	La IA no tiene acceso de escritura; solo sugiere en el chat.

⚠️ **La regla del curso:** la IA es un **copiloto de diagnostico**, no un piloto automatico. Acelera encontrar la causa, pero la decision de que cambio entra al codigo es SIEMPRE de una persona. Self-healing "asistido" (IA propone, humano aprueba) > self-healing "automatico" (IA decide sola).

💡 Esto NO significa que el self-healing sea malo. Significa que debe tener **supervision**. Las herramientas seria del mercado (Testim, Mabl, Healenium) aplican self-healing, pero con reportes de que sanaron para que un humano lo revise. Eso lo vemos en la Parte C.

PARTE C

Self-healing de selectores: Healenium (y como seria en Playwright)

Cierre conceptual: existen herramientas que **sanan selectores rotos solas** (self-healing). La mas conocida es **Healenium**. La vemos como herramienta de mercado para entender el concepto, con honestidad sobre que SI y que NO aplica a nuestro stack.

Seccion 12 · Es demostracion conceptual (Healenium es de Selenium, no de Playwright)

12. Healenium: self-healing de selectores (conceptual)

Que es, como funciona, y por que NO se enchufa directo a nuestro Playwright

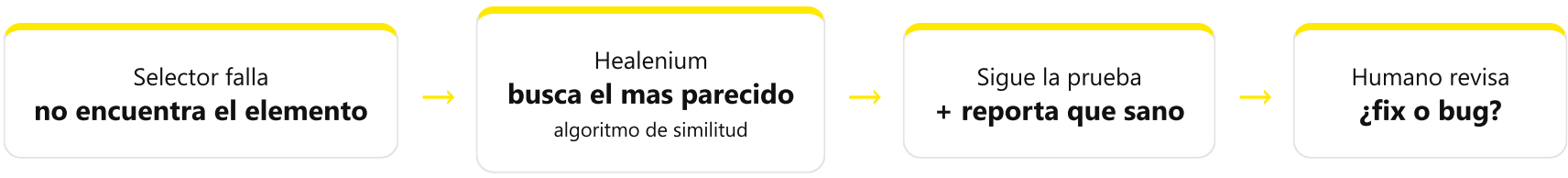
⚠️ **HONESTIDAD TECNICA primero: Healenium es para Selenium** (Java y Python con Selenium WebDriver). Nuestro framework es **Playwright + pytest-bdd**, asi que Healenium **NO se integra directamente** a nuestro proyecto. Lo vemos como **concepto y herramienta de mercado**, no como algo que conectemos hoy.

Que es Healenium

Healenium es una libreria open source que le da **self-healing** a pruebas con Selenium. Cuando un localizador deja de encontrar su elemento (porque el DOM cambio), en vez de fallar de inmediato, Healenium busca el elemento **mas parecido** al original usando un algoritmo de similitud, continua la prueba y **reporta** que localizador "sano" y cual uso en su lugar.

Como funciona (arquitectura, en simple)

1. **Guarda** una "huella" de cada elemento la primera vez que lo encuentra (en una base de datos, PostgreSQL).
2. Cuando un localizador **falla**, compara el DOM actual contra esa huella y calcula el nodo mas similar (arbol de similitud).
3. **Usa** ese nodo para seguir la prueba y **registra** el "healing" en un reporte.
4. Un humano revisa el reporte y decide si actualizar el localizador oficial o si era un bug.



Que necesitaria Healenium (para que veas por que es otro stack)

- **Selenium WebDriver** (no Playwright) como motor de automatizacion.
- Un **servicio Healenium** (backend) y una **base de datos** (PostgreSQL) donde guarda las huellas, normalmente con Docker.
- El **proxy/driver de Healenium** envolviendo tu WebDriver.

Es un stack completo aparte. Por eso en este curso lo mostramos para **conocer el concepto**, no para integrarlo a nuestro Playwright.

¿Como se haria algo parecido EN Playwright? (opcion B, conceptual)

Playwright no tiene Healenium, pero el **mismo concepto** (fallback de selectores) se puede implementar a mano. La idea: una interaction que, si el selector principal falla, intenta **selectores alternativos** (por rol, texto o un data-testid secundario) y registra que uso un fallback.

Idea de self-healing casero en Playwright (pseudo-codigo)

```
def abrir_con_fallback(page, selectores: list[str], descripcion: str):
    """Intenta varios selectores; si uno falla, pasa al siguiente y lo registra."""
    for i, sel in enumerate(selectores):
        try:
            page.wait_for_selector(sel, timeout=3000)
            if i > 0:
                print(f"[self-healing] '{descripcion}': uso el fallback {sel} "
                      f"(el principal {selectores[0]} fallo). REVISAR.")
            return page.locator(sel)
        except TimeoutError:
            continue
    raise AssertionError(f"Ningun selector encontro '{descripcion}': {selectores}")

# Uso: primero el data-testid oficial, luego alternativas estables
grid = abrir_con_fallback(page, [
    '[data-testid="grid-productos"] .card',   # el oficial
    '.grid .card',                            # fallback por clase
    'main .card',                             # fallback por estructura
], descripcion="grid de productos")
```

⚠️ **Mismo principio, misma precaucion:** este fallback **avisa** ("REVISAR") en vez de callar. Nunca debe tapar un cambio silenciosamente — por los riesgos de la seccion 11. Self-healing SI, pero con reporte y revision humana.

📘 DOCUMENTACION DE REFERENCIA

Self-healing — referencia

- **Healenium:** self-healing para **Selenium**; guarda huellas en PostgreSQL y usa similitud de arbol DOM para hallar el elemento cuando el selector cambia.
- **Mercado:** Testim y Mabl ofrecen self-healing propietario con reportes de healing (vistos en el Dia 4 como comparativa).
- **En Playwright:** no hay Healenium; se emula con **fallback de locators** + registro, o se apoya el diagnostico en IA (Claude + MCP), como en la Parte B.

Fuentes: Healenium (sitio oficial) · Healenium (GitHub) · Playwright Locators.
Contenido reformulado para cumplir con las restricciones de licencia.

A tener en cuenta y puente al Dia 6

Lo que construiste y hacia donde va

Lo que construiste hoy

- El framework del Dia 4 ahora corre en **navegadores reales en Sauce Labs**, sin reescribir pruebas.
- **Cross-browser** en los tres motores en local (chromium/firefox/webkit), con una **matriz** en el pipeline (chromium y firefox) y **paralelismo** con pytest-xdist.
- Evidencias ricas: **Allure**, **trazas de Playwright** y video en Sauce.
- **IA en el CI/CD**: triaje automatico de fallos que arma el prompt para Claude + MCP.
- Un **azure-pipelines.yml** con matriz de navegadores, secretos y publicacion de resultados.

Capa	Herramienta	Corre de verdad
E2E + BDD	Playwright + pytest-bdd (Screenplay)	Si (local)
Multi-navegador	Chromium / Firefox / WebKit	Si (local)
Paralelismo	pytest-xdist (`-n auto`)	Si (local)
Reportes / evidencia	Allure + trazas Playwright	Si (local)
IA en CI/CD	triage_ia.py + Claude/MCP	Si (script); IA = Claude
Ejecucion en nube	Sauce Labs (navegadores reales)	Demo guiada (requiere cuenta)
CI/CD	Azure Pipelines (matriz)	Local-first + demo

Puente al Dia 6

Manana entramos a **contenedores y contract testing**: contenerizamos la app Tienda con **Docker** (la misma que hoy probamos en la nube) y validamos los contratos entre el front y la API con **Pact**. La app que empaquetamos sera la que ya conoces de punta a punta.

Documentacion oficial por tema:

Playwright en Sauce | Playwright pytest | pytest-xdist | Trace Viewer | Allure | Variables secretas (Azure)